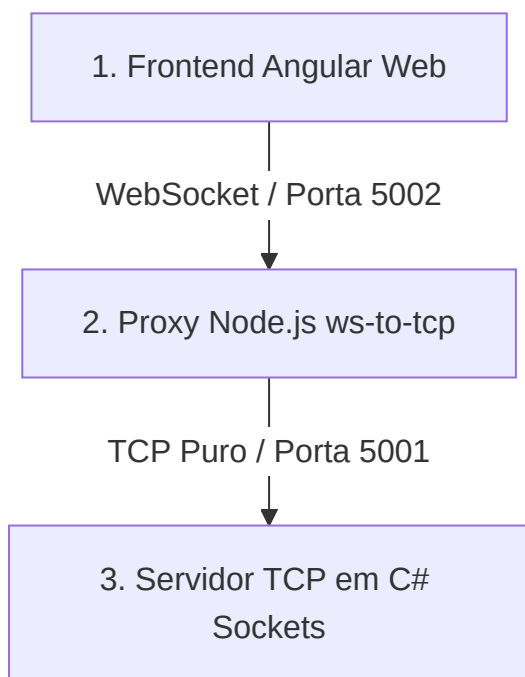


Arquitetura e Fluxo de Implementação: Jogo da Velha TCP / WebSocket

Este documento detalha o fluxo de dados ponto a ponto da aplicação do **Jogo da Velha Multiplayer**, cobrindo o trajeto desde a interação do usuário na interface web até o processamento das regras de negócio no servidor de sockets em C#, passando pela camada intermediária do Proxy em Node.js.

1. Visão Geral da Arquitetura

Devido a restrições de segurança dos navegadores modernos (que proíbem conexões TCP brutas diretas via JavaScript por questões de Sandbox), a aplicação foi desenhada em 3 camadas complementares:



Tecnologias Utilizadas:

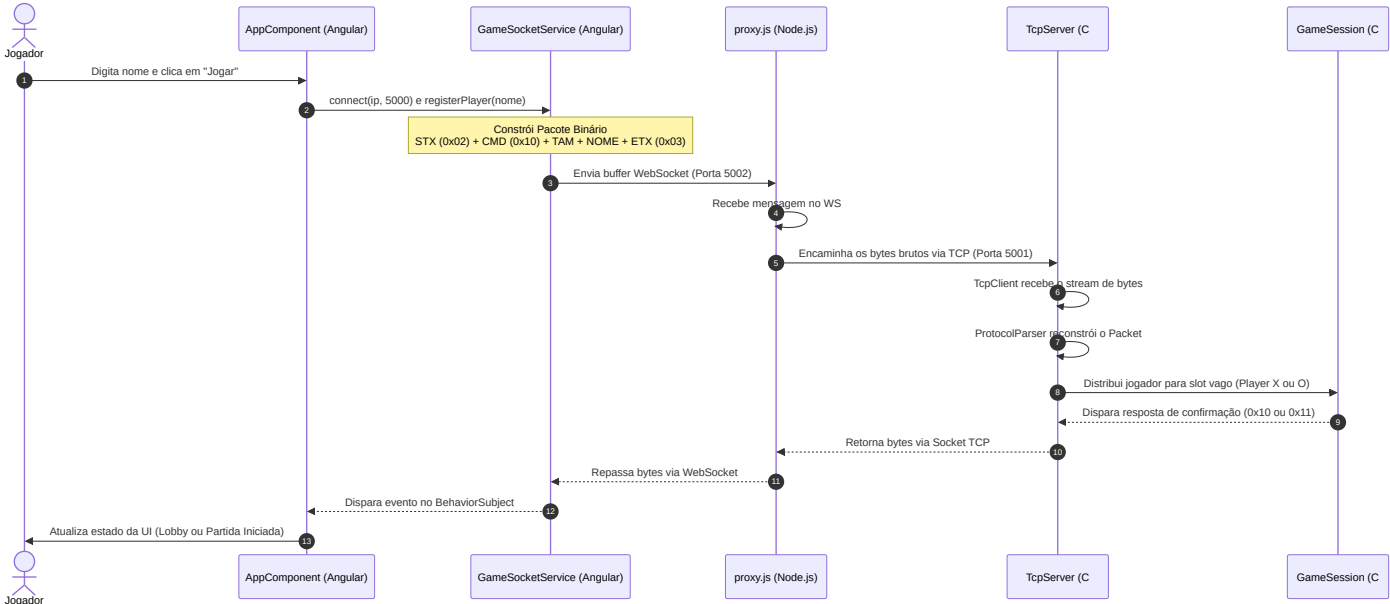
- **Frontend:** Angular 16+ com comunicação reativa baseada em RxJS.
- **Intermediário:** Node.js com a biblioteca `ws` e módulo nativo `net` (TCP).
- **Backend:** C# .NET Core utilizando `TcpListener`, `TcpClient` e programação assíncrona `thread-safe`.

2. Fluxo Detalhado de uma Ação do Usuário

Para entender o funcionamento, acompanhe a jornada de duas ações fundamentais: **Registrar o Jogador** e **Realizar uma Jogada no Tabuleiro**.

Fluxo A: Registro de Jogador (Lobby)

Quando o usuário preenche seu nome na tela inicial e clica em **"Jogar"**:



Detalhes do Código:

1. Interface (app.component.ts):

Chama o método `connectAndRegister()` que ativa o socket e subscreve no estado de conexão.

```
this.socketService.connect(this.serverIp);
this.socketService.registerPlayer(this.playerName);
```

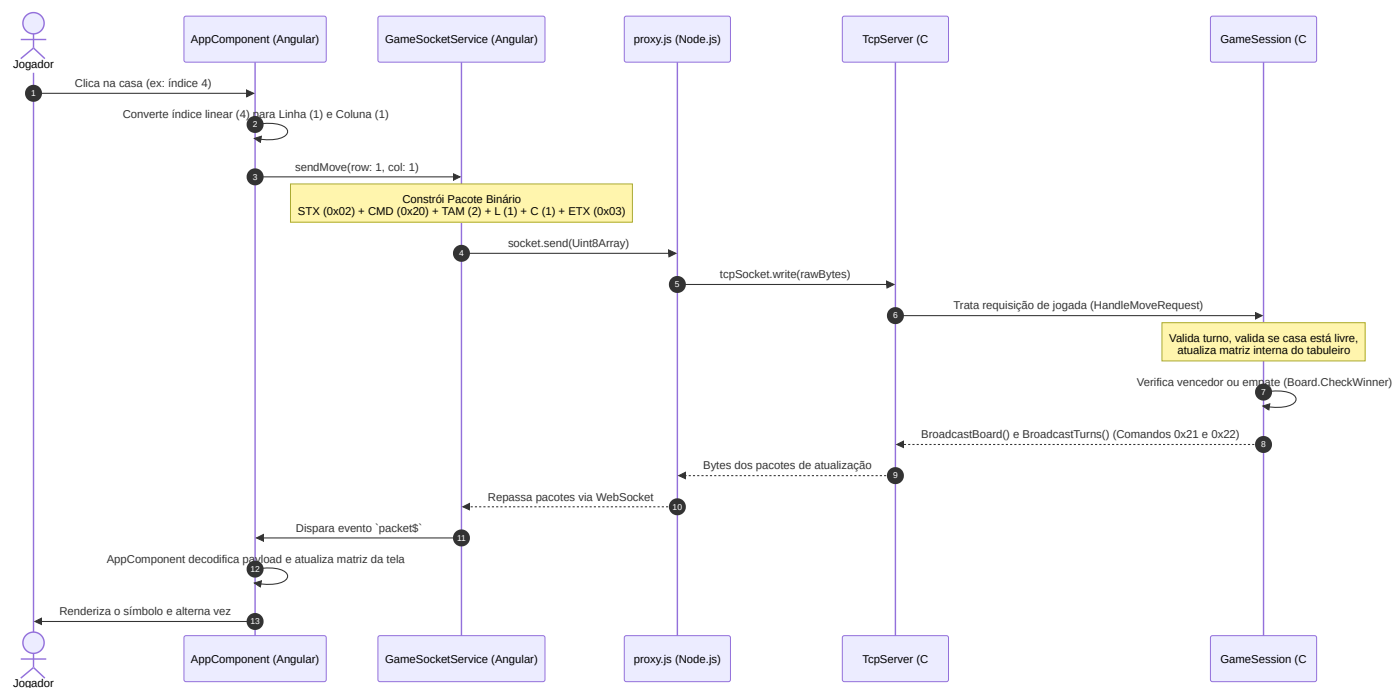
2. Formatação Binária (game-socket.service.ts):

O nome do jogador é codificado em UTF-8 e encapsulado com delimitadores de controle de transmissão física.

```
const encoder = new TextEncoder();
const nameBytes = encoder.encode(name);
const packet = new Uint8Array(4 + nameBytes.length);
packet[0] = 0x02; // STX (Start of Text)
packet[1] = 0x10; // Tipo de Comando: REGISTER_CMD
packet[2] = nameBytes.length; // Comprimento
packet.set(nameBytes, 3); // Carga útil (Payload)
packet[3 + nameBytes.length] = 0x03; // ETX (End of Text)
this.send(packet);
```

Fluxo B: Realização de uma Jogada (Turno Ativo)

Quando o jogador clica em uma das 9 casas vazias do tabuleiro durante seu turno:



Detalhes do Código:

1. Conversão de Coordenadas (app.component.ts):

A tela exibe um array linear de 9 elementos (índices 0 a 8). O protocolo do backend exige o formato bidimensional (Matriz 3x3). Fazemos a conversão:

```
const row = Math.floor(index / 3); // Linha (0, 1 ou 2)
const col = index % 3;             // Coluna (0, 1 ou 2)
this.socketService.sendMove(row, col);
```

2. Formatação Binária (game-socket.service.ts):

Gera um array de bytes compacto de 6 bytes:

```
[ 0x02, 0x20, 0x02, row, col, 0x03 ]
```

```
const moveBytes = new Uint8Array([0x02, 0x20, 0x02, row, col, 0x03]);
this.send(moveBytes);
```

3. O Papel do Intermediário: proxy.js

O arquivo proxy.js atua na camada de transporte. Ele não altera o conteúdo das mensagens, apenas converte o protocolo de encapsulamento da rede física de entrada (WebSocket) para a de saída (TCP puro).

Lógica de Ponte Bidirecional

- **Escuta WebSocket (Entrada):** Porta 5002 .
- **Conecta TCP (Saída):** Porta 5001 (Servidor em C#).

```
// Quando o navegador se conecta ao proxy via WebSocket
wss.on('connection', (ws, req) => {
  const tcpSocket = new net.Socket();

  // Estabelece canal TCP com o servidor C#
  tcpSocket.connect(5001, '127.0.0.1', () => {
    console.log('[PROXY] Conectado ao servidor TCP principal');
  });

  // Repasse: Frontend (WS) -> Backend (TCP)
  ws.on('message', (message) => {
    if (tcpSocket.writable) {
      tcpSocket.write(Buffer.from(message));
    }
  });

  // Repasse: Backend (TCP) -> Frontend (WS)
  tcpSocket.on('data', (data) => {
    if (ws.readyState === WebSocket.OPEN) {
      ws.send(data);
    }
  });

  // Tratamento de queda para evitar conexões órfãs
  ws.on('close', () => tcpSocket.destroy());
  tcpSocket.on('close', () => ws.close());
});
```

4. O Servidor Backend em C# (TcpServer.cs & GameSession.cs)

A Máquina de Estados do ProtocolParser.cs

Como o TCP é um protocolo orientado a fluxo de bytes contínuo (*stream*), os dados enviados podem sofrer fragmentação ou concatenação na rede física. O C# utiliza uma classe especializada baseada em buffer dinâmico e marcadores de limite de mensagem (STX/ETX):

1. O servidor recebe bytes da rede e os adiciona ao acumulador `_buffer` .
2. Busca o primeiro byte `0x02` (STX). Se não achar, limpa o buffer.

3. Se achou `0x02`, garante que possui no mínimo 3 bytes para ler o cabeçalho (`STX` + `TYPE` + `LENGTH`).
4. Lê o byte de `LENGTH` e calcula o tamanho esperado total do frame: `3 + LENGTH + 1` (`ETX`).
5. Se o buffer tem menos bytes que o necessário, aguarda novas leituras de rede.
6. Valida se o último byte na posição calculada é o `0x03` (`ETX`). Se não for, descarta o início desalinhado.
7. Havendo sucesso, extrai a carga útil e dispara o evento do pacote processado, limpando a região correspondente no buffer acumulador.

Gerenciamento de Quedas de Conexão e Timeout de W.O.

No arquivo `GameSession.cs`, existe um algoritmo concorrente thread-safe para tratar a reconexão dos jogadores:

- Se a conexão física com o jogador cair, o servidor dispara um timer assíncrono de tolerância de **15 segundos** (`CancellationTokenSource`).
- Se o jogador reconectar dentro do prazo informando o seu símbolo correspondente, o token é cancelado, a sessão é reassumida e o estado do tabuleiro e a vez de jogo são retransmitidos.
- Se o tempo expirar sem reconexão, o servidor decreta a vitória por W.O. ao jogador remanescente, envia o comando `0x40` de encerramento da partida e encerra os sockets limpando os recursos.